

XYRATECH

XyraSpeech

Complete Project Description & Technical Blueprint

A local-first AI speech platform for developers, creators, and applications.

TEXT-TO-SPEECH • SPEECH-TO-TEXT • VOICE CLONING

Ollama-powered text preparation • Developer API • Voice Studio

MVP target: 3-day implementation sprint

Designed to run locally without paid AI APIs

Prepared as a project description and implementation blueprint. It describes the intended product; it does not claim that the complete software has already been implemented or tested.

1. Executive Summary

XyraSpeech is a proposed speech technology product under XyraTech. It aims to give developers a single, self-hostable platform for generating speech from text, transcribing audio, generating speech in an authorized reference voice, and preparing text for more natural spoken delivery. The platform is designed to run on a developer's computer or on infrastructure controlled by the operator, reducing dependence on paid third-party AI APIs.

The system separates language intelligence from speech synthesis. Ollama optionally prepares text and returns structured delivery hints; dedicated TTS and voice-cloning models generate audio; an STT model converts incoming audio to text. FastAPI exposes these capabilities, and a React web interface provides a usable Voice Studio.

Important: local-first means no paid AI API is required for local development. It does not eliminate hardware, electricity, optional hosting, or future commercial operating costs.

2. Vision, Goals, and Scope

Vision

Make speech-generation and speech-recognition capabilities accessible through a developer-friendly platform that can be run locally, tested without usage-based AI API charges, and extended as the product matures.

Goals

- Provide local Text-to-Speech (TTS) and Speech-to-Text (STT).
- Offer authorized reference-voice cloning through a compatible model, subject to license and hardware checks.
- Use Ollama to normalize punctuation and phrasing and produce optional emotion/delivery metadata.
- Expose documented HTTP endpoints and an interactive API reference.
- Provide a web interface for text input, audio upload/recording, playback, and downloads.
- Keep text enhancement optional so developers can preserve exact wording.

Out of scope for the 3-day MVP

- Training a foundation speech model from scratch.
- Matching every ElevenLabs feature, quality level, language, latency, or scale.
- Production billing, enterprise SLAs, or globally hosted infrastructure.
- Unrestricted impersonation or cloning voices without permission.

3. Product Capabilities

Module	Purpose	MVP behavior
TTS	Convert supplied text to speech.	Enter text, generate audio, play and download it.
STT	Convert audio to text.	Upload audio and display transcription; microphone capture where supported.
Voice cloning	Generate speech resembling an authorized reference speaker.	Reference-audio workflow through a compatible cloning-capable model.
Ollama Text Intelligence	Prepare text for spoken delivery.	Raw, enhanced, and expressive modes with structured output.
Developer API	Let external apps call capabilities.	FastAPI routes, validation, health checks, OpenAPI docs.
Voice Studio	Human-friendly testing interface.	TTS/STT tabs, audio controls, transcription, status and errors.
Local operations	Run without paid AI APIs.	Environment configuration, model setup instructions, tests.

4. Ollama Text Intelligence

Ollama is an optional local language-model runtime, not a TTS engine and not a voice-cloning engine. Its role is to prepare developer-provided text before the text is sent to the speech model.

Mode	Behavior	Recommended use
raw	Returns exact input unchanged; does not call Ollama.	Fixed scripts, exact announcements, legal text.
enhanced	Improves punctuation, capitalization, spacing, and sentence boundaries while preserving meaning.	General-purpose speech generation.
expressive	Splits text into natural segments and adds conservative emotion/delivery hints.	Narration and expressive demos.

Example input: hey john i got the job really excited to tell you

Enhanced text: Hey, John! I got the job! I'm really excited to tell you!

Expressive mode may return metadata such as emotion=excited or delivery=energetic. These are hints, not guaranteed audio effects: they influence sound only when the selected speech engine supports matching controls. Punctuation may guide phrasing but cannot guarantee emotional delivery.

Reliability rules

- Do not add facts, names, promises, or claims absent from the input.
- Preserve numbers, URLs, proper nouns, and the original language.
- Use structured JSON output and validate it against a schema.
- Return original and prepared text so the caller can review changes.
- Never rewrite text in raw mode.

5. System Architecture

The architecture keeps the UI, API orchestration, and model runtimes separate. Each speech capability can be developed and tested independently.

Layer	Responsibilities
React + Vite Voice Studio	Text editor, audio upload/recording, playback, downloads, transcript display.
FastAPI application	Request validation, routing, error handling, health checks, API documentation.
Application services	Text-preparation service, TTS adapter, STT adapter, voice-cloning adapter.
Local model runtimes	Ollama for text intelligence; selected TTS/voice model; faster-whisper or compatible STT model.
Local files/config	Temporary audio, settings, optional history, environment variables.

Typical workflows

TTS: Developer or UI → optional Ollama text preparation → TTS engine → audio file/stream → caller or browser.

STT: Microphone or uploaded audio → validation/conversion → STT engine → transcript → caller or browser.

Voice cloning: Authorized reference recording + text → cloning-capable TTS model → generated audio. Ollama is optional and only prepares text.

Optional AI voice conversation: microphone → STT → Ollama generates a response → optional text preparation → TTS → playback. This is separate from basic TTS/STT.

6. Proposed Technology Stack

Layer	Candidate	Notes
Backend	Python + FastAPI + Pydantic	Async HTTP endpoints, validation, OpenAPI docs.
LLM text prep	Ollama + local chat model	Model size depends on Mac memory and performance.
STT	faster-whisper	Local transcription; benchmark language quality and latency.
TTS / cloning	OpenVoice V2 plus a compatible TTS base model, subject to verification	Check code, weight and dependency licenses and language coverage.
Frontend	React + Vite	Voice Studio interface.
Audio tools	FFmpeg; soundfile or equivalent	Audio conversion and inspection.
Testing	pytest + httpx / FastAPI TestClient	Unit tests and API contract tests.
Packaging	Python venv or optional Docker	A local Python setup may be simplest for the first Mac MVP.

These are candidate technologies, not a claim that every combination is already integrated. Confirm Mac hardware, model compatibility, language needs, and licensing before locking versions.

7. Proposed API Contract

Endpoint	Purpose	Input/output
GET /health	API status.	Returns service status.
GET /health/ollama	Check Ollama availability.	Returns model availability and installed models.
POST /api/v1/text/prepare	Normalize or annotate text.	Text, mode, language, preserve_words; returns prepared text and optional segments/SSML.
POST /api/v1/tts	Generate speech.	Text, voice/model settings; returns audio or a file reference.
POST /api/v1/stt	Transcribe audio.	Audio upload and optional language; returns transcript and optional timestamps.
POST /api/v1/voices/clone	Register a permitted voice profile.	Reference audio and consent confirmation; model-dependent.
POST /api/v1/tts/clone	Generate speech using a reference voice.	Text, voice ID and supported settings; returns audio.

Example text-preparation request

```
{
  "text": "hey john i got the job really excited to tell you",
  "mode": "expressive",
  "language": "English",
  "preserve_words": true,
  "include_ssml": false
}
```

The TTS/STT/voice endpoints are proposed contracts. Implement each only after its model adapter is integrated and tested.

8. Data Handling, Privacy, and Security

- Default to local processing and make external network access explicit.
- Treat audio and transcripts as potentially sensitive; do not log full content by default.
- Set file-size, duration, and accepted-format limits for uploads.
- Use temporary files safely and clean them up unless the user explicitly saves output.
- If API keys are added, store only hashed keys and never commit secrets.
- For voice cloning, require confirmation that the user owns the voice or has permission to use it.
- Do not support deceptive impersonation, fraud, or non-consensual cloning.
- Disclose synthetic speech where appropriate.

9. Three-Day MVP Delivery Plan

Day	Tasks	Exit criteria
Day 1 — Engines	Check Mac hardware; set up Python; install Ollama and pull a suitable model; implement text preparation; run TTS/STT models independently.	Text-preparation endpoint works; one local TTS and one STT path run on samples.
Day 2 — Integration	Build FastAPI routes/adapters; build React pages; connect text preparation to TTS; add upload/transcription and playback.	User can prepare text, generate speech, upload audio, and view transcript.
Day 3 — Reliability	Add validation, clear errors, health checks, tests, setup docs, performance checks, and demo.	Clean setup succeeds; core flows pass a repeatable checklist; limitations documented.

Scope control

Three days is an aggressive deadline. The achievable target is a demonstrable local MVP, not a production-scale ElevenLabs replacement. If cloning is too slow or incompatible with the machine, deliver TTS, STT, and Ollama text preparation first, and keep cloning disabled until its end-to-end test passes.

10. Testing and Acceptance Criteria

- Health endpoint reports whether Ollama is reachable and whether the configured model is installed.
- Raw mode returns exact input and does not call Ollama.
- Enhanced mode returns non-empty text and valid structured output.
- Expressive mode returns validated segments with allowed emotion/delivery values.
- Caller can compare original and prepared text before synthesis.
- TTS produces playable audio from a representative sample.
- STT returns a transcript from a representative audio sample.
- Invalid files, oversized uploads, missing models, and timeouts return clear errors.
- No paid AI API credentials are needed for the local demo.
- Voice cloning is tested only with an authorized voice.

Suggested demo

Enter an informal sentence → choose Enhanced or Expressive → review prepared text and emotion hints → generate speech → play/download audio → upload a short recording → view its transcript → demonstrate the same operations through the API docs.

11. Risks and Mitigations

Risk	Impact	Mitigation
Mac resource limits	Slow inference or memory pressure.	Use smaller models, short samples, sequential inference, and benchmark first.
Model license incompatibility	Commercial release may be restricted.	Review code, weights, dependencies and datasets separately.
Unsupported emotion controls	Metadata may not affect audio.	Map hints only to controls supported by the selected engine.
LLM changes meaning	Incorrect or unwanted speech.	Raw mode, conservative prompt, before/after review, schema validation.
Language mismatch	Poor Tamil/Tanglish or other language quality.	Test representative samples; do not promise unsupported languages.
Voice misuse	Privacy, impersonation, or legal harm.	Consent checks, access controls, and clear acceptable-use rules.

12. Local Setup and Operations

Recommended setup sequence

- Install a supported Python version and Node.js; create a virtual environment.
- Install and start Ollama, then pull a model that fits the Mac.
- Install backend dependencies and configure OLLAMA_BASE_URL and OLLAMA_MODEL.
- Start FastAPI and verify /health and /health/ollama.
- Install frontend dependencies and start the Vite development server.
- Run API tests, then test TTS/STT with short sample inputs.

Keep model weights out of Git unless redistribution terms and repository size are appropriate. Provide model-download instructions instead.

13. Future Roadmap

Phase	Potential additions
MVP	Local TTS, STT, Ollama text preparation, Voice Studio, documented API.
Next	Streaming synthesis where supported, saved voice profiles, Tamil and other language evaluation, timestamps, SDK examples.
Productization	Authentication, API-key management, quotas, monitoring, user-level storage, packaging.
Commercial scale	Benchmarking, deployment options, cost model, reliability controls, license review, support planning.
Advanced speech	Fine-tuning or training with licensed data and sufficient compute; richer prosody and voice design.

14. Commercial and Licensing Checklist

- Verify licenses for the TTS model, voice-cloning model, model weights, and every dependency.
- Confirm commercial use, redistribution, attribution, and model-specific restrictions.
- Do not assume an open-source code license automatically covers weights or training data.
- Measure latency, memory use, transcription quality, and audio quality on target hardware.
- Calculate future hosting and compute costs before offering a paid hosted API.
- Check product name, domain, and trademark availability before public launch.

15. Summary

XyraSpeech is planned as a modular, local-first speech platform: Ollama provides optional text intelligence, dedicated speech models provide TTS/STT and authorized voice cloning, FastAPI exposes developer interfaces, and React provides the Voice Studio. The three-day target is a demonstrable end-to-end MVP with transparent limitations, reproducible setup, and no paid AI API dependency.

Document status: Project description and implementation blueprint. Exact model versions, hardware performance, language coverage, and commercial license compatibility must be validated during implementation.